Übungsblatt 2: Scheduling-Theorie und kooperativer Dispatcher

Musterlösung · Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Aufgabe 1: Ein Prüfauftrag auf der Zeitachse (6 Punkte)

Ein periodischer Prüfauftrag hat $T = 15\,\text{ms}$ und $D = 10\,\text{ms}$. Der betrachtete Job wird bei $r = 3\,\text{ms}$ freigegeben. Er rechnet von 4 bis 6 ms, wird durch andere Arbeit verdrängt und rechnet von 9 bis 12 ms weiter. Danach ist er fertig. Er wartet weder auf Daten noch auf ein Betriebsmittel.

1. Zeichnen Sie Freigabe, Rechenstücke, Fertigstellung, absolute Deadline und nächste Freigabe auf einer Zeitachse ein. (2 Punkte)
2. Berechnen Sie eigene CPU-Zeit, gesamte Antwortzeit und Wartezeit auf Prozessorzuteilung. Wird die Deadline eingehalten? (3 Punkte)
3. Dürfen Sie die beobachtete CPU-Zeit dieses einen Jobs als sichere WCET aller Prüfaufträge übernehmen? Begründen Sie. (1 Punkt)

Musterlösung: Freigabe bei 3, Rechenstücke $[4, 6)$ und $[9, 12)$, Fertigstellung bei 12, absolute Deadline $d = r + D = 13$ und nächste Freigabe bei $r + T = 18\,\text{ms}$.

Die tatsächlich benötigte CPU-Zeit ist $c = 2 + 3 = 5\,\text{ms}$. Die Antwortzeit beträgt $R = f - r = 12 - 3 = 9\,\text{ms}$. Ready ohne Zuteilung ist der Job in $[3, 4)$ und $[6, 9)$, insgesamt 4 ms. Damit $R = 5 + 4 = 9\,\text{ms} \leq D$. Die Deadline ist eingehalten. Ein verdrängter Job ist hier Ready, nicht Blocked.

WCET bedeutet Worst-Case Execution Time. Ein beobachteter Bedarf von 5 ms beweist keine obere Bedarfsschranke C für alle Eingaben und Ausführungspfade. Bewertung gemäß Teilaufgaben; Antwortzeit und eigene CPU-Zeit müssen unterschieden werden.

Aufgabe 2: Welcher feste Rang passt? (6 Punkte)

Für die folgenden periodischen, unabhängigen Tasks gilt $D_i = T_i$. Ein einzelner Kern arbeitet vollständig präemptiv; Blockierungen, Selbstsuspendierungen und Wechselkosten fehlen. C_i ist jeweils eine gegebene sichere CPU-Bedarfsschranke.

Task einer Verpackungsstation	C_i in ms	T_i in ms
Versanddaten	4	20
Antriebskorrektur	3	8
Positionsprüfung	1	5

1. Ordnen Sie die RMS-Prioritäten zu. RMS steht für Rate Monotonic Scheduling. (1 Punkt)

2. Berechnen Sie $U = \sum_i C_i/T_i$. (2 Punkte)
3. Prüfen Sie die hinreichende Liu-Layland-Schranke $U \leq n(2^{1/n} - 1)$. Welche Aussage ist damit für dieses Modell bewiesen? Was würde eine Überschreitung der Schranke bedeuten? (3 Punkte)

Musterlösung: Positionsprüfung > Antriebskorrektur > Versanddaten, da kürzere Periode einen höheren festen Rang erhält.

$$U = \frac{1}{5} + \frac{3}{8} + \frac{4}{20} = \frac{31}{40} = 0{,}775 = 77{,}5\,\%.$$

Für drei Tasks ist $U_{\mathrm{LL}} = 3(2^{1/3} - 1) \approx 0{,}779763$. Damit ist das Set unter den genannten Voraussetzungen mit präemptivem RMS fristgerecht ausführbar. Der Test ist hinreichend: Eine Überschreitung würde lediglich bedeuten, dass dieser Test nicht entscheidet. Sie bewiese weder einen Deadline-Miss noch allgemeine Unmöglichkeit. Die Rechnung ist kein Nachweis für einen kooperativen Funktionsaufrufring.

Aufgabe 3: Über der Schranke genauer prüfen (8 Punkte)

Eine andere Station hat die folgenden Tasks. Es gelten dieselben Modellannahmen wie in Aufgabe 2, insbesondere $D_i = T_i$.

Task	C_i in ms	T_i in ms
Messwerterfassung	2	6
Stellwertberechnung	3	12
Chargenbericht	5	24

1. Berechnen Sie Auslastung und RMS-Prioritätsfolge. Entscheidet die Liu-Layland-Schranke? (2 Punkte)
2. Bestimmen Sie für jede Task die Worst-Case-Antwortzeit. Beginnen Sie mit $R_i^{(0)} = C_i$ und rechnen Sie bis zum Fixpunkt oder einer Fristüberschreitung: (5 Punkte)

$$R_i^{(k+1)} = C_i + \sum_{h \in hp(i)} \left\lceil \frac{R_i^{(k)}}{T_h} \right\rceil C_h.$$

$hp(i)$ bezeichnet die höher priorisierten Tasks.

3. Warum wird aufgerundet und warum darf eine Iteration nicht bereits beim ersten Wert unterhalb der Deadline beendet werden? (1 Punkt)

Musterlösung: Messwerterfassung > Stellwertberechnung > Chargenbericht.

$$U = \frac{2}{6} + \frac{3}{12} + \frac{5}{24} = \frac{19}{24} \approx 79{,}17\,\% > 77{,}98\,\%.$$

Der hinreichende Test entscheidet nicht. Die Antwortzeitanalyse ergibt:

Task	Iteration in ms	Fixpunkt / relative Deadline
Messwerterfassung	$2 \to 2$	$2 \leq 6$

Task	Iteration in ms	Fixpunkt / relative Deadline
Stellwertberechnung	$3 \to 5 \to 5$	$5 \leq 12$
Chargenbericht	$5 \to 10 \to 12 \to 12$	$12 \leq 24$

Für den Bericht lauten die drei Updates:

$$\begin{aligned} 5 + \lceil 5/6 \rceil \cdot 2 + \lceil 5/12 \rceil \cdot 3 &= 10, \\ 5 + \lceil 10/6 \rceil \cdot 2 + \lceil 10/12 \rceil \cdot 3 &= 12, \\ 5 + \lceil 12/6 \rceil \cdot 2 + \lceil 12/12 \rceil \cdot 3 &= 12. \end{aligned}$$

Alle Tasks halten im Modell ihre Fristen. Die Aufrundung zählt vollständige höher priorisierte Jobs, die im betrachteten Antwortzeitfenster interferieren können. Ein zunächst kleiner Wert kann das Fenster vergrößern und weitere Freigaben hineinnehmen. Erst der Fixpunkt schließt diese Rückwirkung ab. Fünf Punkte für die drei korrekten Iterationen und die Fristentscheidung; zwei für Rang, Auslastung und Schrankenentscheidung; einer für die Begründung.

Aufgabe 4: RMS und EDF entscheiden verschieden (6 Punkte)

Zwei unabhängige periodische Tasks starten synchron bei null. Es gilt das präemptive Ein-Kern-Modell aus Aufgabe 2 und jeweils $D_i = T_i$.

Task	C_i in ms	T_i in ms
Ventilregelung	3	6
Bildauswertung	4	9

1. Berechnen Sie die Auslastung. Was folgt unter diesen Voraussetzungen für EDF, Earliest Deadline First? (1 Punkt)
2. Zeichnen Sie RMS von 0 bis 10 ms. Markieren Sie den ersten Fristverstoß und den Restbedarf zum Deadlinezeitpunkt. (2 Punkte)
3. Zeichnen Sie EDF von 0 bis 18 ms und beschriften Sie die absoluten Jobdeadlines. Bei gleichen Deadlines läuft der bereits laufende Job weiter. Warum verdrängt die Freigabe bei 6 ms die laufende Task hier nicht? (3 Punkte)

Zusatz: Zwei andere Tasks haben $(C, T, D) = (3, 24, 4)\,\text{ms}$ und $(2, 30, 4)\,\text{ms}$, beide mit erster Freigabe bei null. Ihre Auslastung liegt unter 20 Prozent. Können trotzdem schon die ersten Jobs unter jeder Auswahlregel scheitern? Begründen Sie über den Bedarf bis zur ersten Deadline.

Musterlösung: $U = 3/6 + 4/9 = 17/18 \approx 94{,}44\,\% \leq 100\,\%$. EDF ist im vollständig präemptiven Modell mit unabhängigen Tasks und impliziten Deadlines fristgerecht möglich.

RMS priorisiert die Ventilregelung. Sie läuft 0–3, dann die Bildauswertung 3–6, dann wieder die Ventilregelung 6–9. Bei der Bilddeadline 9 ms fehlt dem ersten Bildjob noch 1 ms. Er endet erst bei 10 ms. Auch die Antwortzeititeration $4 \to 7 \to 10 > 9$ zeigt die Fristverletzung.

EDF-Intervall in ms	Job	absolute Deadline in ms
$[0, 3)$	Ventilregelung, Job 1	6
$[3, 7)$	Bildauswertung, Job 1	9
$[7, 10)$	Ventilregelung, Job 2	12
$[10, 14)$	Bildauswertung, Job 2	18

EDF-Intervall in ms	Job	absolute Deadline in ms
$[14, 17)$	Ventilregelung, Job 3	18
$[17, 18)$	Idle	—

Bei 6 ms hat der neue Ventiljob die Deadline 12 ms. Der laufende Bildjob hat weiterhin die frühere Deadline 9 ms und läuft deshalb weiter. Bei 12 ms erhält der neue Ventiljob dieselbe Deadline 18 ms wie der laufende Bildjob; die vorgegebene Gleichstandsregel lässt diesen weiterlaufen. Alle bis 18 ms fälligen Jobs enden rechtzeitig. Die allgemeine EDF-Aussage folgt aus dem Test unter seinen Voraussetzungen, nicht allein aus diesem endlichen Ausschnitt.

Zusatz: $U = 3/24 + 2/30 = 23/120 \approx 19{,}17\,\%$. Bis zur gemeinsamen Deadline bei 4 ms benötigen die beiden ersten Jobs jedoch $3 + 2 = 5\,\text{ms}$ CPU-Zeit. Mindestens einer verfehlt seine Frist. Es gilt hier $D_i < T_i$; der einfache EDF-Auslastungstest aus dem Hauptteil ist nicht anwendbar.

Aufgabe 5: Aufrufreihenfolge ist keine Zeitscheibe (6 Punkte)

Ein kooperativer Dispatcher ruft ständig bereite Schritte in der Reihenfolge **Signal, Prüfen, Anzeige** auf. Der erste Signal-Schritt beginnt bei null. Signal benötigt 1 ms CPU-Zeit, Anzeige 2 ms. Alle Verwaltungs- und Interruptkosten sind im Modell null; jeder Schritt kehrt zurück. Es gibt hier keine periodischen Freigaben.

1. Prüfen benötigt stets 2 ms. Bestimmen Sie die ersten vier Signal-Starts und die CPU-Anteile der drei Schritte pro Runde. (2 Punkte)
2. Prüfen benötigt abwechselnd 2 und 5 ms, zuerst 2 ms. Bestimmen Sie die ersten fünf Signal-Starts sowie Minimum, Maximum und Spanne der Startabstände. Ändert sich allein deshalb die eigene CPU-Zeit von Signal? (3 Punkte)
3. Prüfen kehrt durch einen Programmfehler nicht mehr zurück. Werden Signal und Anzeige erneut aufgerufen? Kann ein zugelassener Timerinterrupt trotzdem laufen? (1 Punkt)

Musterlösung: Bei konstantem Bedarf dauert eine Runde $1 + 2 + 2 = 5\,\text{ms}$. Signal startet bei 0, 5, 10, 15 ms. Die Anteile betragen $1/5 = 20\,\%$, $2/5 = 40\,\%$, $2/5 = 40\,\%$. Gleiche Aufrufzahlen sind keine gleichen CPU-Zeitanteile.

Bei wechselnder Prüfarbeit dauern die Runden abwechselnd 5 und 8 ms. Signal startet bei 0, 5, 13, 18, 26 ms. Die Startabstände haben Minimum 5, Maximum 8 und Spanne $8 - 5 = 3\,\text{ms}$. Die eigene Signal-Arbeit bleibt 1 ms; ihre Wiederaufrufabstände schwanken.

Ohne Rückkehr aus Prüfen erreicht der Kontrollfluss keine weiteren Schritte. Ein nicht gesperrter Timerinterrupt kann trotzdem laufen und die Zeit fortschreiben. Dadurch entsteht noch kein Taskwechsel.

Aufgabe 6: Eine Signalfolge selbst implementieren

Programmieren, etwa 25–35 Minuten. Nutzen Sie die vorbereitete **NUCLEO-H755ZI-Q / STM32H755ZIT6**-Basis mit STM32CubeMX und VS Code mit STM32 VS Code Extension: CM4 ohne RTOS, generierter Dual-Core-Start. Die Projektvorbereitung erfolgt vor dem Termin.

Implementieren Sie eine wiederholte Doppelpulsanzeige: **120 ms ein, 180 ms aus, 120 ms ein, 780 ms aus**. Gleichzeitig müssen ein zweiter LED-Schritt und ein kurzer Zählschritt weiter ausgeführt werden.

Bearbeiten Sie den leeren Rumpf von `void Lab02_SignalStep(void)` in `signal_step.c` aus `code/lab02/`. Dispatcher, Hauptschleife und Begleitschritte sind bereits eingebunden.

Bereits vorhandene Schnittstelle	Wirkung
`Lab02_SignalWrite(on)`	Signal-LED setzen: null aus, sonst ein
`Lab02_SignalPause(ms)`	aktiven Schritt bis zum Wake-Ziel sperren; beendet die Funktion nicht

1. Skizzieren Sie die vier Zustände der Signalfolge. Welche Information muss zwischen zwei Aufrufen erhalten bleiben?
2. Implementieren Sie pro Aufruf eine LED-Aktion, speichern Sie die nächste Phase, setzen Sie die Pause und **kehren Sie zurück**. Kein `HAL_Delay`, keine Warteschleife oder eigene Endlosschleife im Schritt.
3. Bauen und laden Sie das Projekt. Prüfen Sie Signalfolge, laufende Status-LED und Fortschritt von `lab02_signal_checks`: zwei Zählerstände bei angehaltenem CM4 mit einem Lauf dazwischen vergleichen.
4. Notieren Sie die idealen ersten fünf Flankenzeitpunkte ab dem ersten Einschalten. Warum garantiert die 1200-ms-Summe keine exakte physische Periode?
5. Erweitern Sie auf drei Pulse mit je 100 ms Ein- und Zwischen-Auszeit sowie 900 ms Schluss-Auszeit. Prüfen Sie erneut den Fortschritt der anderen Schritte.

Ein Messgerät ist nicht erforderlich. Halten Sie fest, ob Ihre Funktion auf dem Board geprüft wurde.

Quellen: Buttazzo, *Hard Real-Time Computing Systems*; Liu und Layland (1973); ST UM2408 und RM0399. Die Aufgabenwerte sind Lehrannahmen.

Musterlösung zu Aufgabe 6

Musterlösung: Die Phase muss über Funktionsrückkehr hinweg erhalten bleiben, beispielsweise als statische Variable. Es gibt keinen pausierten Stack pro kooperativem Schritt.

```
#include "lab02_signal.h"

void Lab02_SignalStep(void)
{
    static uint32_t phase = 0U;
    static const uint32_t duration_ms[4] = {120U, 180U, 120U, 780U};
    const uint32_t delay_ms = duration_ms[phase];

    Lab02_SignalWrite((phase & 1U) == 0U);
    phase = (phase + 1U) % 4U;
    Lab02_SignalPause(delay_ms);
    return;
}
```

Ideale Flanken: 0 ms (ein), 120 ms (aus), 300 ms (ein), 420 ms (aus), 1200 ms (ein). Das Wake-Ziel stellt nur Bereitschaft her. Tickphase, andere Schritte und Auswahl verzögern die Ausführung; die Pause wird nach der LED-Aktion gesetzt. Ein exaktes physisches Raster oder eine sichere Mindestdauer jeder Phase folgt daraus nicht.

Für drei Pulse: `duration_ms[6] = {100U, 100U, 100U, 100U, 100U, 900U}` und Modulo 6. Gerade Phasen schalten ein, die ideale Wiederholung dauert 1400 ms. Status wartet jeweils 700 Ticks, der Zählschritt kehrt nach einem Inkrement zurück. Nach der Pausenhilfe muss auch der Signal-Schritt enden, damit andere Schritte laufen können.

Prüfkriterien: persistente Phasen, passende LED-Zustände und Pausen, begrenzte Arbeit, Fortschritt aller Schritte. Das sichtbare Muster beweist keine WCET oder Fristgarantie.